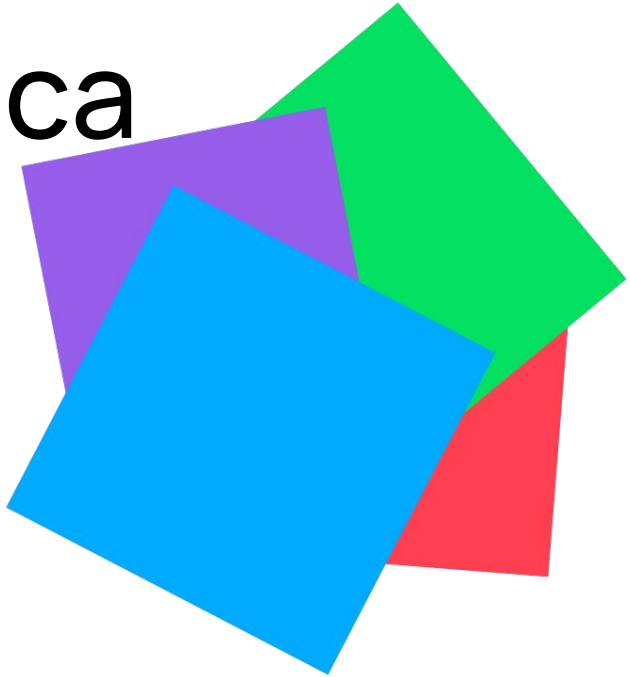
Занятие 6.

Архитектура сервиса



Один сценарий — публикация объявления — пройдём от свалки до слоёв

1. Разберём, что такое слои, богатые модели и порты - на одном коде

2. Увидим живой рефакторинг: было - стало, с тестами до и после

3. Честно поговорим, когда архитектура - это оверинжиниринг



Avito Start

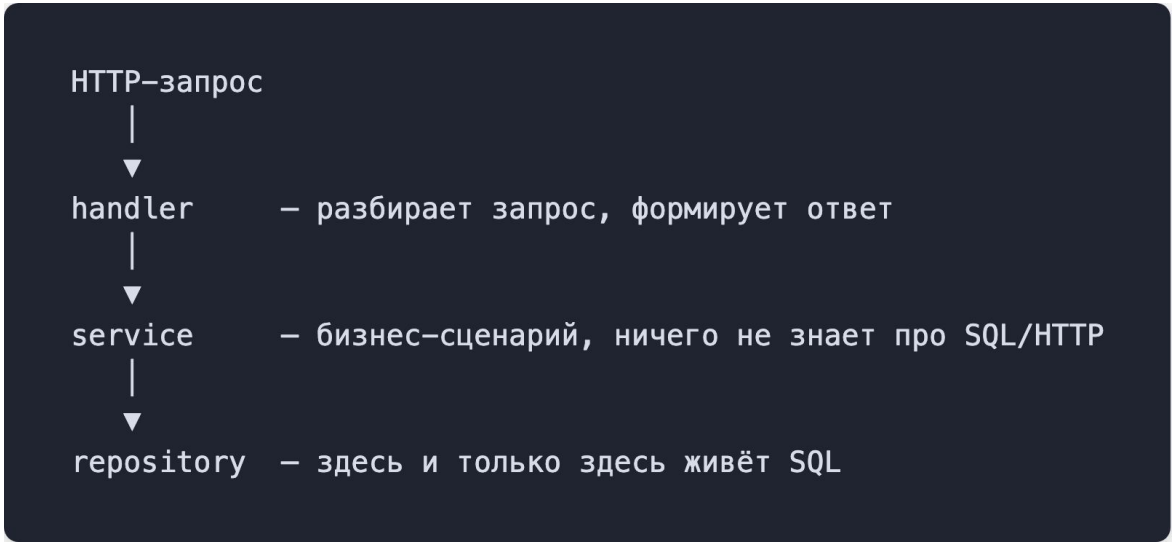
Сервис, который работает, но пугает

```
func PublishListingHandler(db *sql.DB, modURL string) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        // разбор запроса
        // бизнес-правило: нет цены
        // бизнес-правило: нет фото
        // INSERT в БД
        // HTTP-вызов в модерацию
        // решение, что ответить клиенту
    }
}
```

before/handler.go целиком

```go
func PublishListingHandler(db *sql.DB, moderationURL string) http.HandlerFunc {
  return func(w http.ResponseWriter, r *http.Request) {
    var req PublishListingRequest
    json.NewDecoder(r.Body).Decode(&req)
    if req.Price <= 0 {
      http.Error(w, "price must be positive", 400); return
    }
    if strings.TrimSpace(req.PhotoURL) == "" {
      http.Error(w, "photo is required", 400); return
    }
    var id int64
    db.QueryRowContext(r.Context(),
      `INSERT INTO listings (...) RETURNING id`, ...).Scan(&id)
    modReq, _ := http.NewRequestWithContext(r.Context(),
      "POST", moderationURL+"/check", ...)
    resp, err := http.DefaultClient.Do(modReq)
    if err != nil {
      // модерация недоступна — объявление уже в БД,
      // решаем тут же, по месту: отвечаем 201 pending
      w.WriteHeader(201)
      json.NewEncoder(w).Encode(map[string]any{"id": id, "status": "pending"})
      return
    }
    if resp.StatusCode == 200 {
      db.ExecContext(r.Context(), `UPDATE listings SET status=$1 WHERE id=$2`, "approved", id)
    }
    w.WriteHeader(201); json.NewEncoder(w).Encode(...)
  }
}
```

Слои и границы ответственности

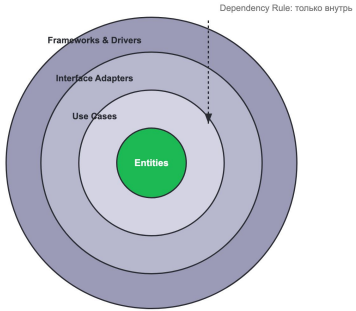


Архитектурные стили - сравнение



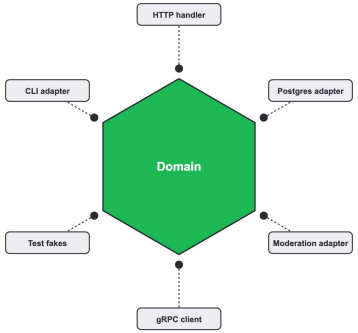
Слои идут друг за другом линейно: запрос проходит от UI к данным и обратно

N-tier (слоёная)



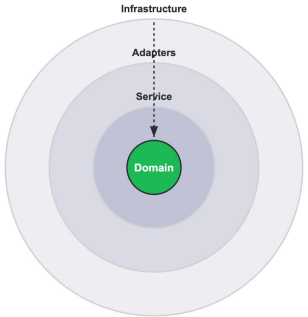
Р. Мартин, 2012 — 4 именованных кольца, та же идея направления зависимостей

Clean Architecture (R. Martin)



Порты на границе домена, адаптеры подключаются снаружи

Hexagonal / Ports & Adapters (A. Cockburn)



Кольца вокруг домена, зависимости — только внутрь

Onion (J. Palermo)

Идеи и названия принадлежат авторам: J. Palermo (Onion, 2008), R. Martin (Clean Architecture, 2012), A. Cockburn (Hexagonal / Ports and Adapters, 2005).

Avito Start

Живой рефакторинг: разносим свалку по слоям

Демо на экране: берём before/handler.go и прямо на паре растаскиваем на четыре файла.

```
after/
├── domain/      Listing — инварианты, переходы статусов
├── service/     ListingService.Publish — сценарий + порты
├── repository/  Postgres- и HTTP-адаптеры
└── handler/     JSON ↔ доменные ошибки ↔ HTTP-статусы
```

Код на экране, не в слайдах — репозиторий открыт параллельно.

after/domain/listing.go

```go
type Status string
const (
  StatusPending  Status = "pending"
  StatusApproved Status = "approved"
  StatusRejected Status = "rejected"
)

type Listing struct {
  ID int64; Title string; Price float64
  PhotoURL string; SellerID int64
  Status Status; CreatedAt time.Time
}

// единственный способ создать Listing — инварианты проверены сразу
func NewListing(title string, price float64,
  photoURL string, sellerID int64) (*Listing, error) {
  if price <= 0 { return nil, ErrInvalidPrice }
  if strings.TrimSpace(photoURL) == "" { return nil, ErrPhotoMissing }
  return &Listing{..., Status: StatusPending, CreatedAt: time.Now()}, nil
}

// поведение, а не смена поля — модель сама решает, откуда можно перейти
func (l *Listing) Approve() error {
  if l.Status != StatusPending {
    return errors.New("only pending can be approved")
  }
  l.Status = StatusApproved; return nil
}
```

after/service/ports.go

```go
// порт для хранилища — service зависит от интерфейса, не от *sql.DB
type ListingRepository interface {
  Create(ctx context.Context, l *domain.Listing) (int64, error)
  UpdateStatus(ctx context.Context, id int64, status domain.Status) error
  Get(ctx context.Context, id int64) (*domain.Listing, error)
}

// порт для модерации — сегодня HTTP, завтра может стать очередью
type Moderator interface {
  Check(ctx context.Context, listingID int64) (approved bool, err error)
}
```

after/service/listing_service.go

```go
func (s *ListingService) Publish(ctx context.Context,
  in PublishInput) (*domain.Listing, error) {
  listing, err := domain.NewListing(in.Title, in.Price, in.PhotoURL, in.SellerID)
  if err != nil { return nil, err } // доменная ошибка прокинута выше

  id, err := s.repo.Create(ctx, listing)
  if err != nil { return nil, err }
  listing.ID = id

  approved, err := s.moderator.Check(ctx, id)
  if err != nil {
    // модерация недоступна — явное решение: остаётся pending
    return listing, nil
  }

  if approved { listing.Approve() } else { listing.Reject() }
  s.repo.UpdateStatus(ctx, id, listing.Status)
  return listing, nil
}
```

after/repository/ - адаптеры

```go
type PostgresListingRepository struct { db *sql.DB }

func (r *PostgresListingRepository) Get(ctx context.Context,
  id int64) (*domain.Listing, error) {
  var l domain.Listing
  err := r.db.QueryRowContext(ctx, `SELECT ... WHERE id = $1`, id).
    Scan(&l.ID, &l.Title, &l.Price, &l.PhotoURL, &l.SellerID, &l.Status, &l.CreatedAt)

  if errors.Is(err, sql.ErrNoRows) {
    return nil, domain.ErrNotFound // перевод sql-ошибки в доменную
  }
  if err != nil { return nil, err }
  return &l, nil
}

// HTTPModerator — второй адаптер, реализует service.Moderator
func (m *HTTPModerator) Check(ctx context.Context,
  listingID int64) (bool, error) {
  resp, err := m.client.Do(req)
  if err != nil { return false, err }
  return resp.StatusCode == http.StatusOK, nil
}
```

after/handler/listing_handler.go

```go
func (h *ListingHandler) Publish(w http.ResponseWriter, r *http.Request) {
  var req publishRequest
  json.NewDecoder(r.Body).Decode(&req)

  listing, err := h.service.Publish(r.Context(), service.PublishInput{...})
  if err != nil { writeError(w, err); return }

  w.WriteHeader(http.StatusCreated)
  json.NewEncoder(w).Encode(listing)
}

// единственное место, которое решает HTTP-статус по доменной ошибке
func writeError(w http.ResponseWriter, err error) {
  switch {
  case errors.Is(err, domain.ErrInvalidPrice), errors.Is(err, domain.ErrPhotoMissing):
    http.Error(w, err.Error(), http.StatusBadRequest)
  case errors.Is(err, domain.ErrNotFound):
    http.Error(w, err.Error(), http.StatusNotFound)
  default:
    http.Error(w, "internal error", http.StatusInternalServerError)
  }
}
```

Модели: анемичные и богатые

```go
// анемичная — структура-мешок,
// вся логика в service
type Listing struct {
    Status string
}

// service.go:
if l.Status != "pending" { ... }
l.Status = "approved"
```

```go
// богатая — метод сам
// проверяет инвариант
func (l *Listing) Approve() error {
  if l.Status != StatusPending {
    return errors.New("...")
  }
  l.Status = StatusApproved
  return nil
}
```

Ошибки: от SQL до HTTP-статуса

```
// repository: переводим sql.ErrNoRows в доменную ошибку
if errors.Is(err, sql.ErrNoRows) {
    return nil, domain.ErrNotFound
}

// handler: доменную ошибку — в HTTP-статус
case errors.Is(err, domain.ErrNotFound):
    http.Error(w, err.Error(), http.StatusNotFound)
```

Порты и адаптеры (гексагональная архитектура)

```
// порт — интерфейс в service
type ListingRepository interface {
    Create(ctx, *Listing) (int64, error)
    Get(ctx, int64) (*Listing, error)
}

// адаптер — конкретная реализация в repository
type PostgresListingRepository struct { db *sql.DB }
```

Знакомство с DDD

Bounded context - «Объявление» в контексте публикации и «Объявление» в контексте модерации - не одно и то же: важны разные свойства.

Агрегат — группа объектов, которая меняется только вместе и только через одну точку входа (у нас — через Listing).

Когда слои - это оверинжиниринг

```
before/handler.go   —  107 строк
after/  (4 пакета)  —  347 строк
```

Код вырос почти в 3 раза. Это честная цена - не бесплатный бонус.

Чистая архитектура - инструмент для сложной бизнес-логики, которая будет жить и меняться годами.
Для CRUD-ручки из одного запроса три слоя - лишняя индирекция, а не правило «всегда делай так».

Проверить можно без базы и без сети

```
func TestPublish_ModerationDown_StaysPending(t *testing.T) {
    repo := &fakeRepo{}
    svc := service.NewListingService(repo,
        &fakeModerator{err: context.DeadlineExceeded})

    listing, _ := svc.Publish(ctx, input)
    // listing.Status == StatusPending
}
```

Тест целиком: fakeRepo + fakeModerator

```
type fakeRepo struct { created *domain.Listing }
func (f *fakeRepo) Create(_ context.Context, l *domain.Listing) (int64, error) {
  f.created = l; return 42, nil
}
func (f *fakeRepo) Get(_ context.Context, id int64) (*domain.Listing, error) {
  if f.created == nil || f.created.ID != id { return nil, domain.ErrNotFound }
  return f.created, nil
}

func TestPublish_ModerationDown_StaysPending(t *testing.T) {
  repo := &fakeRepo{}
  svc := service.NewListingService(repo, &fakeModerator{err: context.DeadlineExceeded})

  listing, err := svc.Publish(ctx, service.PublishInput{
    Title: "lamp", Price: 500, PhotoURL: "http://x/1.jpg", SellerID: 1,
  })

  if listing.Status != domain.StatusPending {
    t.Errorf("want pending when moderation is down, got %s", listing.Status)
  }
}
```

Итоги занятия

01

Разнесли код-свалку по слоям

02

Увидели богатую модель и порты

03

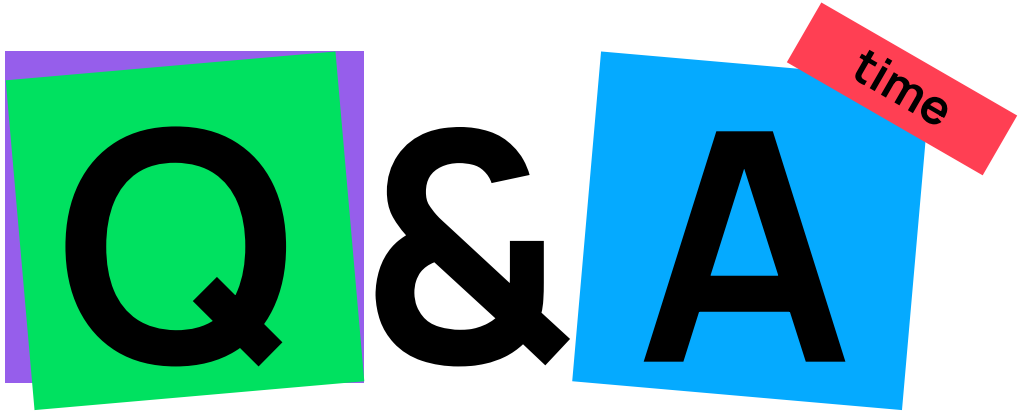
Протестировали бизнес-логику без БД

04

Поняли, когда архитектура лишняя

05

Дальше: практика - отрефакторить такой же сервис


Q&A
time